# BÀI GIẢNG 08: LÝ THUYẾT ĐỒ THỊ & THUẬT TOÁN ĐƯỜNG ĐI

Môn học: Toán Rời Rạc (CO2003) | Giảng viên: TS. Nguyễn Văn Phúc | Tài liệu lưu hành nội bộ

**Tóm tắt bài học:**

## 1. ĐỊNH NGHĨA VÀ BẬC CỦA ĐỈNH TRONG ĐỒ THỊ VÔ HƯỚNG

Một đồ thị vô hướng G được xác định bởi cặp tập hợp G = (V, E), trong đó V là tập các đỉnh (vertices) khác rỗng và E là tập các cạnh (edges) nối các cặp đỉnh. Bậc của đỉnh v trong G, ký hiệu deg(v), là số cạnh liên thuộc với v, với quy ước khuyên (loop) đóng góp 2 vào bậc của đỉnh.

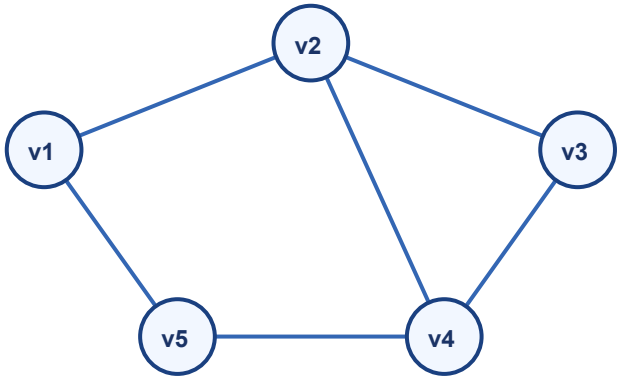
**Định lý 1.1 (Bổ đề bắt tay — Handshaking Lemma):**

Cho đồ thị vô hướng G = (V, E) có |V| đỉnh và |E| cạnh. Khi đó:

$$\sum \deg(v) = 2 \cdot |E| \quad (v \in V)$$

Hệ quả 1.1: Trong mọi đồ thị vô hướng, số lượng đỉnh có bậc lẻ luôn là một số chẵn. Chứng minh: Gọi V_chẵn và V_lẻ lần lượt là tập đỉnh bậc chẵn và bậc lẻ. Tổng bậc của toàn bộ đỉnh bằng 2|E| (số chẵn), mà tổng bậc các đỉnh thuộc V_chẵn là chẵn, do đó tổng bậc các đỉnh thuộc V_lẻ phải là một số chẵn, suy ra |V_lẻ| là số chẵn.

**Minh họa 1.1: Đồ thị mẫu G1 với 5 đỉnh và 6 cạnh**




**Bảng bậc của các đỉnh:**

| Đỉnh | Đỉnh kề | deg(v) |
|---|---|---|
| v1 | {v2, v5} | 2 |
| v2 | {v1, v3, v4} | 3 |
| v3 | {v2, v4} | 2 |
| v4 | {v2, v3, v5} | 3 |
| v5 | {v1, v4} | 2 |

**Tổng bậc: ∑ deg(v) = 12 = 2 · 6 (Thỏa Định lý 1.1)**

1.2. Biểu diễn đồ thị trên máy tính:
Có hai cấu trúc dữ liệu chính được sử dụng phổ biến:
a) Ma trận kề A = [a_ij] kích thước n × n, trong đó a_ij = 1 nếu (v_i, v_j) ∈ E và a_ij = 0 nếu ngược lại. Ma trận kề đồ thị vô hướng luôn là ma trận đối xứng (A = A^T). Ưu điểm: kiểm tra liên thuộc O(1), nhược điểm: tốn bộ nhớ O(n^2), không hiệu quả cho đồ thị thưa (sparse graph).
b) Danh sách kề (Adjacency List): mỗi đỉnh v duy trì một danh sách các đỉnh kề với nó. Bộ nhớ yêu cầu là O(|V| + |E|), duyệt đỉnh kề tối ưu O(deg(v)).

# 2. THUẬT TOÁN TÌM ĐƯỜNG ĐI NGẮN NHẤT — DIJKSTRA

Thuật toán Dijkstra (Edsger W. Dijkstra, 1959) giải quyết bài toán tìm đường đi ngắn nhất từ một đỉnh nguồn s đến tất cả các đỉnh còn lại trong đồ thị có trọng số không âm w(u, v) ≥ 0. Thuật toán duy trì ước lượng khoảng cách d[v] từ s tới v và tập S các đỉnh đã xác định được khoảng cách tối ưu.

**Mã giả thuật toán Dijkstra (Sử dụng hàng đợi ưu tiên Min-Priority Queue):**

```
function Dijkstra(G = (V, E, w), s):
    for each vertex v in V:
        d[v] = ∞;  parent[v] = null
    d[s] = 0
    Q = PriorityQueue()  // Khởi tạo hàng đợi nhị phân min-heap
    Q.insert(s, 0)
    while not Q.isEmpty():
        u = Q.extractMin()  // Đỉnh có d[u] nhỏ nhất chưa thăm
        for each neighbor v of u with weight w(u, v):
            if d[u] + w(u, v) < d[v]:  // Bước nới lỏng (Relaxation)
                d[v] = d[u] + w(u, v)
                parent[v] = u
                Q.decreaseKey(v, d[v])
    return d, parent
```

### Bảng phân tích độ phức tạp thời gian theo cấu trúc dữ liệu:

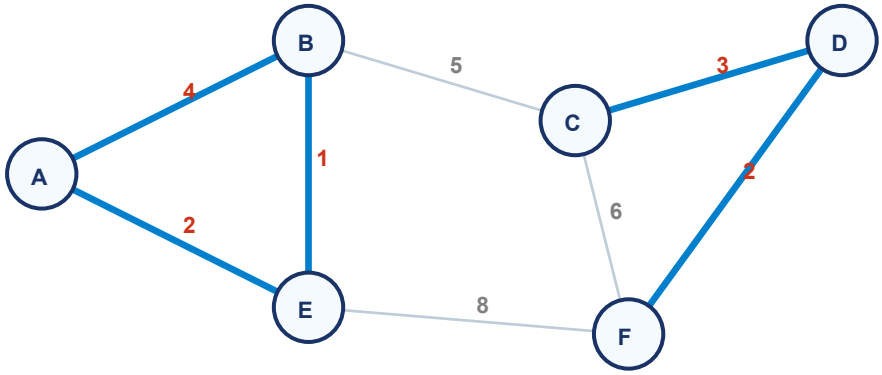
| Cấu trúc hàng đợi Q | Tìm min (Extract-Min) | Nới lỏng (Decrease-Key) | Tổng thời gian T(V, E) |
|---|---|---|---|
| Mảng tuần tự (Array) | O(V) | O(1) | **$O(V^2 + E) = O(V^2)$** |
| Nhị phân (Binary Heap) | O(log V) | O(log V) | **O((V + E) log V)** |
| Heap d-phân (d-ary Heap) | O(d log_d V) | O(log_d V) | **O((d V + E) log_d V)** |
| Fibonacci Heap | O(log V) amortized | O(1) amortized | **O(E + V log V)  [Tối ưu lý thuyết]** |

Cảnh báo quan trọng về trọng số âm:
Thuật toán Dijkstra KHÔNG hoạt động chính xác khi đồ thị tồn tại cạnh có trọng số âm (w(u, v) < 0). Nguyên nhân: nguyên lý tham lam (Greedy Choice Property) giả định rằng khi một đỉnh u được lấy ra khỏi Q với khoảng cách d[u] nhỏ nhất, không thể có đường đi nào khác qua các đỉnh chưa thăm có thể làm giảm d[u]. Giả định này sai lầm nếu có trọng số âm. Để giải bài toán với trọng số âm và phát hiện chu trình âm, bắt buộc phải sử dụng thuật toán Bellman-Ford với độ phức tạp O(V · E).

# 3. CÂY KHUNG NHỎ NHẤT (MINIMUM SPANNING TREE - MST)

Cho đồ thị vô hướng liên thông có trọng số G = (V, E, w). Một cây khung T = (V, E_T) là một đồ thị con liên thông không có chu trình chứa tất cả các đỉnh của G (|E_T| = |V| - 1). Cây khung nhỏ nhất (MST) là cây khung có tổng trọng số w(T) = ∑_{e □ E_T} w(e) đạt giá trị nhỏ nhất.

**Sơ đồ mạng tô-pô có trọng số & Cây khung tối ưu:**




**Chú thích: Các cạnh đậm màu xanh dương thuộc về cây khung nhỏ nhất T (Tổng trọng số: w(T) = 2 + 1 + 4 + 3 + 2 = 12)**

**Bảng diễn tiến từng bước theo thuật toán Kruskal (Sắp xếp cạnh tăng dần):**

| Bước | Cạnh (u, v) | Trọng số w | Kiểm tra tạo chu trình (Disjoint-Set) | Quyết định |
|---|---|---|---|---|
| 1 | **(B, E)** | 1 | Find(B) ≠ Find(E) -> Hợp nhất {B, E} | **CHẤP NHẬN** |
| 2 | **(A, E)** | 2 | Find(A) ≠ Find(E) -> Hợp nhất {A, B, E} | **CHẤP NHẬN** |
| 3 | **(D, F)** | 2 | Find(D) ≠ Find(F) -> Hợp nhất {D, F} | **CHẤP NHẬN** |
| 4 | **(C, D)** | 3 | Find(C) ≠ Find(D) -> Hợp nhất {C, D, F} | **CHẤP NHẬN** |
| 5 | **(A, B)** | 4 | Find(A) = Find(B) = tập {A, B, E} -> Tạo chu trình A-B-E | **LOẠI BỎ** |
| 6 | **(B, C)** | 5 | Find(B) ≠ Find(C) -> Hợp nhất 2 thành phần liên thông | **CHẤP NHẬN (Đủ 5 cạnh)** |

# 4. ỨNG DỤNG THỰC TIỄN & BÀI TẬP TỰ LUYỆN

4.1. Định tuyến gói tin trong mạng Internet (OSPF & IS-IS):
Giao thức Open Shortest Path First (OSPF) là một giao thức định tuyến trạng thái liên kết (Link-State Routing Protocol) được triển khai rộng rãi trong các hệ thống mạng doanh nghiệp và ISP. Mỗi router trong vùng OSPF xây dựng một cơ sở dữ liệu trạng thái liên kết (LSDB) đồng nhất, mô tả toàn bộ tô-pô mạng dưới dạng một đồ thị có trọng số. Mỗi router sau đó độc lập thực thi thuật toán Dijkstra để tính toán cây đường đi ngắn nhất (Shortest Path Tree) từ chính nó tới tất cả các mạng con đích, từ đó nạp bảng chuyển tiếp gói tin (FIB).

4.2. Giao thức chống bão broadcast (Spanning Tree Protocol - STP / RSTP):
Trong mạng cục bộ Ethernet (Layer 2), việc tạo vòng lặp dự phòng cáp dẫn đến thảm họa broadcast storm và MAC flapping. STP (IEEE 802.1D) và RSTP (IEEE 802.1w) sử dụng biến thể phân tán của thuật toán tìm cây khung để tự động khóa các cổng dự phòng, đảm bảo mạng chỉ có một topo hình cây logic duy nhất tại mọi thời điểm.

**BÀI TẬP VỀ NHÀ (Nộp trước 23:59 Chủ Nhật tuần thứ 8):**

Bài 1 (2.0 điểm): Chứng minh rằng trong đồ thị vô hướng liên thông G = (V, E) có trọng số các cạnh đôi một khác nhau, cây khung nhỏ nhất (MST) là duy nhất.

Bài 2 (3.0 điểm): Cho đồ thị có hướng G có n đỉnh. Thiết kế thuật toán kiểm tra sự tồn tại của chu trình có trọng số trung bình nhỏ nhất với độ phức tạp O(n · m).

Bài 3 (2.5 điểm): Giả sử chi phí của mỗi liên kết mạng biến thiên theo hàm thời gian c(e, t) = α_e + β_e · t với t ≥ 0. Nếu khởi hành từ s tại thời điểm t = 0, hãy chứng minh điều kiện cần và đủ để thuật toán Dijkstra mở rộng vẫn đảm bảo tính tối ưu toàn cục.

Bài 4 (2.5 điểm): Cài đặt cấu trúc dữ liệu Disjoint-Set với tối ưu nén đường đi (Path Compression) và hợp nhất theo hạng (Union by Rank) bằng Python hoặc C++, đo đếm số phép toán thực tế.